HTML Complete Notes — Beginner to Intermediate

Table of Contents

1. HTML Home
2. HTML Introduction
3. HTML Editors
4. HTML Basic Structure
5. HTML Elements
6. HTML Attributes
7. HTML Headings
8. HTML Paragraphs
9. HTML Styles
10. HTML Formatting
11. HTML Quotations
12. HTML Comments
13. HTML Colors
14. HTML CSS
15. HTML Links
16. HTML Images
17. HTML Project
18. HTML Favicon
19. HTML Page Title
20. HTML Tables
21. HTML Lists
22. Practice Questions
23. Important Viva Questions
24. HTML Block & Inline
25. HTML Div

26. HTML Classes
27. HTML ID
28. HTML Buttons
29. HTML Iframes
30. HTML JavaScript
31. HTML File Paths
32. HTML Head
33. HTML Layout
34. HTML Responsive
35. HTML Computercode
36. HTML Semantics
37. HTML Style Guide
38. HTML Entities
39. HTML Symbols
40. HTML Emojis
41. HTML Charsets
42. HTML URL Encode
43. HTML vs. XHTML
44. HTML Forms
45. HTML Form Attributes
46. HTML Form Elements
47. HTML Input Types
48. HTML Input Attributes
49. Input Form Attributes
50. Complete HTML Form Project
51. Practice Questions with Solutions
52. Important Viva Questions

---

1. HTML Home

What is HTML?

HTML stands for **HyperText Markup Language**.

It is the standard markup language used to create and structure web pages. It tells the browser what content exists on a page and how that content is organized.

HTML is a markup language, not a programming language, because it describes content structure rather than providing general-purpose programming logic.

Meaning of HTML

- **HyperText:** Text that can link to other documents or web pages.
- **Markup:** Tags used to describe and structure content.
- **Language:** A defined system of syntax and rules.

What can we create using HTML?

- Headings and paragraphs
- Images and hyperlinks
- Tables and lists
- Forms and input fields
- Navigation menus
- Basic website layouts

HTML, CSS, and JavaScript

Technology	Purpose
HTML	Defines the structure of a web page.
CSS	Controls styling, colors, spacing, and layout.
JavaScript	Adds behavior and interactivity.

Example: HTML creates a button, CSS makes it blue, and JavaScript can make it respond when clicked.

---

2. HTML Introduction

Basic HTML Example

Create a file named `index.html`.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>My First Website</title>
</head>
<body>
    <h1>Hello, World!</h1>
    <p>This is my first HTML page.</p>
</body>
</html>
```

Explanation

- `<!DOCTYPE html>` tells the browser to use modern HTML standards.
- `<html>` is the root element of the document.
- `<head>` contains metadata and document information.
- `<meta charset="UTF-8">` supports a wide range of characters.
- `<title>` defines the browser tab title.
- `<body>` contains the content displayed on the page.
- `<h1>` defines the main heading.
- `<p>` defines a paragraph.

How HTML works

1. Write HTML in a text editor.
2. Save the file with the `.html` extension.
3. Open it in a web browser.
4. The browser parses the HTML and renders the page.

HTML tags are generally not displayed as text. Instead, the browser interprets them to create page elements.

---

3. HTML Editors

An HTML editor is a tool used to write and edit HTML code.

Popular editors

- Visual Studio Code
- Notepad
- Notepad++
- Sublime Text
- Web-based HTML editors

Creating your first HTML file

1. Open your preferred editor.
2. Create a new file.
3. Write the following code.

```
<!DOCTYPE html>
<html>
<head>
    <title>My Page</title>
</head>
<body>
    <h1>Welcome</h1>
    <p>I am learning HTML.</p>
</body>
</html>
```

4. Save the file as `index.html`.
5. Open the file in Chrome, Firefox, or another browser.

Important points

- Use `.html` or `.htm` as the file extension.
- Indentation improves readability.
- Modern editors provide syntax highlighting and error assistance.
- You do not need a server to run a simple HTML file.

---

4. HTML Basic Structure

Every standard HTML page has a basic structure.

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">

    <title>Basic HTML Page</title>
</head>

<body>
    <h1>My Website</h1>
    <p>Welcome to my website.</p>
</body>

</html>
```

Document structure

```
HTML Document
|
+-- DOCTYPE declaration
|
+-- html
    |
    +-- head
    |   |
    |   +-- metadata
    |   +-- title
    |
    +-- body
        |
        +-- heading
        +-- paragraph
        +-- other page content
```

Important terms

DOCTYPE: Declares the document type. In modern HTML, use `<!DOCTYPE html>`.

Root element: The `<html>` element contains the document's HTML content.

Head section: Contains metadata, stylesheets, the title, and other document information.

Body section: Contains the visible page content and interactive elements.

Viewport metadata: Helps the page adapt to different screen sizes.

---

5. HTML Elements

An HTML element consists of an opening tag, content, and usually a closing tag.

Example

```
<p>This is a paragraph.</p>
```

Here:

- `<p>` is the opening tag.
- `This is a paragraph.` is the content.
- `</p>` is the closing tag.
- The complete structure is a paragraph element.

Nested elements

Elements can contain other elements.

```
<div>
    <h2>About Me</h2>
    <p>I am learning <strong>HTML</strong>.</p>
</div>
```

The `<strong>` element is nested inside the paragraph, which is nested inside the `<div>`.

Empty elements

Some HTML elements do not have closing tags.

```
<br>
<hr>
<img src="photo.jpg" alt="A photograph">
```

Examples:

- `<br>` inserts a line break.
- `<hr>` represents a thematic break.

- `<img>` embeds an image.

Block and inline elements

Block elements generally start on a new line and occupy the available width.

Examples: `<div>`, `<p>`, `<h1>`, `<ul>`.

Inline elements generally flow within surrounding text.

Examples: `<span>`, `<a>`, `<strong>`, `<em>`.

CSS can change how elements are displayed.

Common mistake

Incorrect:

```
<p><strong>Hello</p></strong>
```

Correct:

```
<p><strong>Hello</strong></p>
```

Nested elements should be closed in the reverse order in which they were opened.

---

6. HTML Attributes

Attributes provide additional information about HTML elements.

Attributes are normally written in the opening tag.

Syntax

```
<tagname attribute="value">Content</tagname>
```

Example

```
<a href="https://example.com"
   target="_blank"
   rel="noopener noreferrer">
    Visit Example
</a>
```

Explanation:

- `href` specifies the destination.

- `target="_blank"` opens the destination in a new browsing context.
- `rel="noopener noreferrer"` provides additional protection and privacy for the new context.

Common HTML attributes

Attribute	Purpose
`id`	Identifies a unique element within a document.
`class`	Groups elements for CSS or JavaScript.
`style`	Applies inline CSS.
`href`	Specifies a link destination.
`src`	Specifies a resource URL, such as an image.
`alt`	Provides alternative text for an image.
`title`	Provides advisory information, often shown as a tooltip.
`width`	Specifies width where supported.
`height`	Specifies height where supported.
`lang`	Identifies the language of content.

Example using attributes

```
<h1 id="main-heading" class="heading">
    Welcome
</h1>

<p style="color: blue;">
    This paragraph is blue.
</p>

<img src="flower.jpg"
     alt="A red flower"
     width="300"
     height="200">
```

Boolean attributes

Boolean attributes represent true or false states through their presence or absence.

```
<input type="text" required>
<input type="checkbox" checked>
<input type="text" disabled>
```

- `required`: The field must be completed before a normal form submission can succeed.
- `checked`: The checkbox is initially checked.
- `disabled`: The control is disabled.

---

7. HTML Headings

HTML provides six heading levels, from `<h1>` to `<h6>`.

```
<h1>Main Heading</h1>
<h2>Section Heading</h2>
<h3>Subsection Heading</h3>
<h4>Heading Level 4</h4>
<h5>Heading Level 5</h5>
<h6>Heading Level 6</h6>
```

Heading hierarchy

```
h1: Main topic
 |
 +-- h2: Major section
      |
      +-- h3: Subsection
           |
           +-- h4: Smaller section
                |
                +-- h5
                     |
                     +-- h6
```

Important points

- `<h1>` represents the highest-level heading.
- `<h6>` represents the lowest-level heading.
- Headings provide document structure and improve navigation.
- Use heading levels according to meaning, not merely to change text size.
- CSS should be used when you need to control visual size.

Example

```
<h1>Computer Science</h1>

<h2>Programming Languages</h2>
<p>Programming languages are used to write software.</p>

<h3>HTML</h3>
<p>HTML structures web pages.</p>

<h3>Java</h3>
<p>Java is a general-purpose programming language.</p>
```

---

8. HTML Paragraphs

The `<p>` element represents a paragraph.

Syntax

```
<p>Your paragraph text goes here.</p>
```

Example

```
<h1>About HTML</h1>

<p>HTML stands for HyperText Markup Language.</p>

<p>It is used to structure web pages.</p>

<p>Browsers interpret HTML and display its content.</p>
```

Browsers generally display separate paragraphs with spacing between them.

Line breaks

Use `<br>` when a line break is meaningful within the same block of text.

```
<p>
    Prajwal Gadge<br>
    Engineering Student<br>
    India
</p>
```

Horizontal rule

Use `<hr>` for a thematic break between sections.

```
<h2>Chapter One</h2>
<p>Introduction to HTML.</p>

<hr>

<h2>Chapter Two</h2>
<p>Introduction to CSS.</p>
```

Preserving whitespace

HTML generally collapses consecutive spaces and line breaks in ordinary text.

Use `<pre>` when preserving whitespace and line breaks is important.

```
<pre>
Name: Prajwal
Course: Engineering
Subject: HTML
</pre>
```

---

9. HTML Styles

The `style` attribute applies CSS directly to an HTML element.

Syntax

```
<tagname style="property: value;">Content</tagname>
```

Example

```
<h1 style="color: blue;">Welcome</h1>

<p style="font-size: 20px;">
    This paragraph has larger text.
</p>

<p style="background-color: yellow;">
    This paragraph has a yellow background.
</p>
```

Common CSS properties

Property	Purpose
`color`	Changes text color.
`background-color`	Changes background color.
`font-size`	Changes text size.
`font-family`	Changes the typeface.
`text-align`	Aligns text.
`margin`	Controls outer spacing.
`padding`	Controls inner spacing.
`border`	Adds a border.

Complete example

```
<!DOCTYPE html>
<html>
<head>
    <title>Styled Page</title>
</head>
<body>

    <h1 style="color: darkblue; text-align: center;">
        My Website
    </h1>

    <p style="font-size: 18px; color: gray;">
        Welcome to my first styled web page.
    </p>

    <button style="background-color: green; color: white;">
        Click Me
    </button>

</body>
</html>
```

Best practice

Inline CSS is useful for quick examples. For real projects, prefer internal or external stylesheets to make styling reusable and easier to maintain.

---

10. HTML Formatting

HTML provides elements for emphasizing or formatting text.

Important formatting elements

Element	Meaning
`<b>`	Draws attention through bold styling without adding strong importance.
`<strong>`	Indicates strong importance.
`<i>`	Represents text in an alternate voice or mood, or a technical term.
`<em>`	Adds stress emphasis.
`<mark>`	Marks highlighted text.
`<small>`	Represents side comments or fine print.
`<del>`	Represents deleted text.
`<ins>`	Represents inserted text.
`<sub>`	Represents subscript text.
`<sup>`	Represents superscript text.

Examples

```
<p><b>Bold text</b></p>

<p><strong>Important information</strong></p>

<p><i>Alternate voice</i></p>

<p><em>Emphasized words</em></p>

<p><mark>Highlighted text</mark></p>

<p><small>Small print</small></p>

<p>Old price: <del>₹1000</del></p>

<p>New price: <ins>₹800</ins></p>

<p>Water formula: H<sub>2</sub>O</p>

<p>Mathematics: x<sup>2</sup></p>
```

Output understanding

- `H<sub>2</sub>O` displays the 2 below the normal text line.
- `x<sup>2</sup>` displays the 2 above the normal text line.
- `<del>` commonly appears with a strikethrough.
- `<ins>` commonly appears underlined.
- `<strong>` and `<em>` convey meaning as well as commonly affecting visual presentation.

Important distinction

`<b>` and `<strong>` may look similar, but their meanings differ. Similarly, `<i>` and `<em>` are not interchangeable in meaning.

Choose the element based on the purpose of the text.

---

11. HTML Quotations

HTML provides elements for quotations and references.

The `<blockquote>` element

Use `<blockquote>` for a longer quotation from another source.

```
<blockquote cite="https://example.com/article">
    Learning is a continuous process.
</blockquote>
```

The `cite` attribute can specify the source URL. It does not automatically display a citation.

The `<q>` element

Use `<q>` for a short inline quotation.

```
<p>My teacher said,
    <q>Practice makes progress.</q>
</p>
```

Browsers generally add quotation marks around `<q>` content.

The `<abbr>` element

Use `<abbr>` for an abbreviation or acronym.

```
<p>
    <abbr title="HyperText Markup Language">HTML</abbr>
    is used to structure web pages.
</p>
```

The `<address>` element

Use `<address>` for contact information related to a person, organization, or document.

```
<address>
    Contact: support@example.com<br>
    Website: https://example.com
</address>
```

The `<cite>` element

Use `<cite>` for the title of a creative work.

```
<p>
    My favorite book is <cite>Wings of Fire</cite>.
</p>
```

The `<bdo>` element

The `<bdo>` element overrides the direction of text.

```
<p><bdo dir="rtl">Hello World</bdo></p>
```

`dir="rtl"` specifies right-to-left direction.

---

12. HTML Comments

Comments are notes in the HTML source that browsers do not display as page content.

Syntax

```
<!-- This is an HTML comment -->
```

Single-line comment

```
<!-- Main heading -->
<h1>Welcome</h1>
```

Multi-line comment

```
<!--
This page demonstrates:
1. HTML headings
2. HTML paragraphs
3. HTML links
-->

<h1>My Website</h1>
<p>Learning HTML step by step.</p>
```

Temporarily commenting out content

```
<!--
<p>This paragraph is currently commented out.</p>
-->
```

Uses of comments

- Explain sections of code.
- Mark the beginning and end of major sections.
- Leave development notes.
- Temporarily exclude content during testing.

Important warning

Comments are not a security mechanism. Anyone who can view the page source may be able to read them. Never place passwords, API secrets, or private information in HTML comments.

---

13. HTML Colors

HTML elements can be colored using CSS.

The `color` property controls text color, while `background-color` controls the background.

1. Color names

```
<p style="color: red;">Red text</p>
<p style="color: blue;">Blue text</p>
<p style="background-color: yellow;">
    Yellow background
</p>
```

2. HEX colors

HEX colors use hexadecimal notation.

```
<p style="color: #ff0000;">Red text</p>
<p style="color: #0000ff;">Blue text</p>
<p style="color: #008000;">Green text</p>
```

Examples:

- `#FF0000` — red
- `#00FF00` — green
- `#0000FF` — blue
- `#000000` — black
- `#FFFFFF` — white

3. RGB colors

RGB specifies red, green, and blue components, each ranging from 0 to 255.

```
<p style="color: rgb(255, 0, 0);">
    Red text
</p>

<p style="color: rgb(0, 0, 255);">
    Blue text
</p>
```

4. RGBA colors

RGBA adds an alpha component for transparency.

```
<div style="background-color: rgba(255, 0, 0, 0.5);">
    Semi-transparent red background
</div>
```

The alpha value ranges from 0 (fully transparent) to 1 (fully opaque).

5. HSL colors

HSL means Hue, Saturation, and Lightness.

```
<p style="color: hsl(120, 100%, 25%);">
    Dark green text
</p>
```

Complete example

```
<!DOCTYPE html>
<html>
<head>
    <title>HTML Colors</title>
</head>
<body style="background-color: #f0f8ff;">

    <h1 style="color: navy;">Color Demonstration</h1>

    <p style="color: red;">Named color</p>

    <p style="color: #008000;">HEX color</p>

    <p style="color: rgb(128, 0, 128);">
        RGB color
    </p>

    <p style="color: hsl(200, 80%, 40%);">
        HSL color
    </p>

</body>
</html>
```

---

14. HTML CSS

CSS stands for **Cascading Style Sheets**.

CSS controls the presentation and layout of HTML documents.

Three ways to add CSS

1. Inline CSS

CSS is written in the `style` attribute.

```
<p style="color: red;">Hello World</p>
```

Useful for quick demonstrations, but difficult to maintain across a large website.

2. Internal CSS

CSS is written inside a `<style>` element in the document's `<head>`.

```
<!DOCTYPE html>
<html>
<head>
    <title>Internal CSS</title>

    <style>
        body {
            background-color: lightgray;
        }

        h1 {
            color: darkblue;
            text-align: center;
        }

        p {
            font-size: 18px;
        }
    </style>
</head>
<body>

    <h1>Welcome</h1>
    <p>This page uses internal CSS.</p>

</body>
</html>
```

3. External CSS

CSS is stored in a separate `.css` file.

Create two files:

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>External CSS</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <h1>My Website</h1>
    <p>This page uses external CSS.</p>

</body>
</html>
```

style.css

```
body {
    background-color: #f0f0f0;
    font-family: Arial, sans-serif;
}

h1 {
    color: navy;
    text-align: center;
}

p {
    color: #444444;
    font-size: 18px;
}
```

Keep both files in the same directory for this example.

CSS syntax

```
selector {
    property: value;
}
```

Example:

```
p {
    color: blue;
    font-size: 20px;
}
```

- `p` is the selector.
- `color` and `font-size` are properties.
- `blue` and `20px` are values.

Common selectors

```
/* Element selector */
p {
    color: blue;
}

/* Class selector */
.note {
    color: green;
}

/* ID selector */
#main {
    color: red;
}

/* Group selector */
h1, h2, h3 {
    font-family: Arial, sans-serif;
}
```

Matching HTML:

```
<p class="note">This is a note.</p>

<h1 id="main">Main Heading</h1>
```

Why external CSS is useful

- Reuses styles across many pages.
- Separates content from presentation.
- Makes website-wide changes easier.
- Helps keep HTML files cleaner.

---

15. HTML Links

Links allow users to navigate to another page, file, section, or resource.

The `<a>` element creates a hyperlink.

Basic syntax

```
<a href="URL">Link Text</a>
```

1. External link

```
<a href="https://www.wikipedia.org/">
    Visit Wikipedia
</a>
```

2. Open a link in a new tab

```
<a href="https://example.com/"
   target="_blank"
   rel="noopener noreferrer">
    Open Example
</a>
```

3. Internal link to another page

Suppose your directory contains:

```
website/
|
+-- index.html
+-- about.html
```

In `index.html`:

```
<a href="about.html">About Us</a>
```

4. Link to a section of the same page

```
<a href="#contact">Go to Contact</a>

<h2 id="contact">Contact Section</h2>
<p>Contact us for more information.</p>
```

The browser navigates to the element with the matching ID.

5. Email link

```
<a href="mailto:hello@example.com">
    Send an Email
</a>
```

6. Telephone link

```
<a href="tel:+911234567890">
    Call Us
</a>
```

7. Image as a link

```
<a href="https://example.com/">
    <img src="logo.png" alt="Visit Example">
</a>
```

Absolute and relative URLs

Absolute URL: Includes the complete destination URL.

```
<a href="https://example.com/about.html">About</a>
```

Relative URL: Resolves from the current document's location.

```
<a href="about.html">About</a>
```

Important points

- Use descriptive link text.
- Ensure the destination exists.
- Use IDs for in-page navigation.
- Avoid vague text such as "click here" when a meaningful label is possible.

---

16. HTML Images

The `<img>` element embeds an image in a web page.

Syntax

```
<img src="image.jpg" alt="Description of the image">
```

The `<img>` element is a void element and does not need a closing tag.

Example

```
<img src="flower.jpg"
     alt="A red flower"
     width="300"
     height="200">
```

Important attributes

Attribute	Purpose
`src`	Image URL or file path.
`alt`	Text alternative for the image.
`width`	Display width.
`height`	Display height.
`loading`	Can enable lazy loading.

Local images

Directory structure:

```
website/
|
+-- index.html
+-- images/
    |
    +-- flower.jpg
```

HTML:

```
<img src="images/flower.jpg"
     alt="A flower">
```

Images from a URL

```
<img src="https://example.com/images/flower.jpg"
     alt="A flower">
```

The image must be available at the specified URL, and the server must allow it to be loaded.

Responsive image

```
<img src="flower.jpg"
     alt="A flower"
     style="max-width: 100%; height: auto;">
```

This allows the image to shrink to fit its container without distorting its proportions.

Image as a link

```
<a href="index.html">
    <img src="logo.png" alt="Home">
</a>
```

Why is `alt` important?

Alternative text helps users who cannot see the image, including users of screen readers. It also appears when an image cannot be displayed.

For a purely decorative image, use `alt=""`.

Common mistakes

- Incorrect file path.
- Misspelled image filename.
- Missing alternative text.
- Setting width and height that distort the image.
- Assuming an external image URL will always remain available.

---

17. HTML Project

Mini Project: Personal Profile Website

This project combines headings, paragraphs, links, images, lists, and CSS.

Directory structure

```
personal-profile/
|
+-- index.html
+-- style.css
+-- images/
    |
    +-- profile.jpg
```

Place your own image in the `images` directory and name it `profile.jpg`.

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">

    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">

    <title>My Personal Profile</title>

    <link rel="stylesheet" href="style.css">
</head>
<body>

    <header>
        <h1>My Personal Profile</h1>
        <p>Welcome to my website!</p>
    </header>

    <main>
        <section>
            <h2>About Me</h2>

            <img src="images/profile.jpg"
                 alt="My profile photograph"
                 width="180"
                 height="180">

            <p>
                Hello! I am a student learning web development.
                I enjoy programming and building projects.
            </p>
        </section>

        <section>
            <h2>My Skills</h2>

            <ul>
                <li>HTML</li>
                <li>CSS</li>
                <li>Java</li>
                <li>SQL</li>
            </ul>
        </section>
```

```
        <section>
            <h2>My Learning Goals</h2>

            <ol>
                <li>Learn HTML fundamentals.</li>
                <li>Learn CSS styling.</li>
                <li>Learn JavaScript.</li>
                <li>Build a complete website.</li>
            </ol>
        </section>

        <section>
            <h2>Contact Me</h2>

            <p>
                <a href="mailto:hello@example.com">
                    Send me an email
                </a>
            </p>
        </section>
    </main>

    <footer>
        <p>&copy; 2026 My Personal Profile</p>
    </footer>

</body>
</html>
```

style.css

```
body {
    margin: 0;
    font-family: Arial, sans-serif;
    background-color: #f4f6f8;
    color: #222222;
    line-height: 1.6;
}

header {
    background-color: #123456;
    color: white;
    text-align: center;
    padding: 25px;
}

main {
    max-width: 800px;
    margin: 20px auto;
    padding: 20px;
}

section {
    background-color: white;
    margin-bottom: 20px;
    padding: 20px;
    border-radius: 8px;
}

img {
    max-width: 100%;
    height: auto;
    border-radius: 8px;
}

a {
    color: #0759a5;
}

footer {
    text-align: center;
    padding: 15px;
    background-color: #dddddd;
}
```

What you learn from this project

- Basic document structure.
- Semantic HTML elements.
- External CSS.
- Images and alternative text.
- Ordered and unordered lists.
- Email links.
- Basic layout and responsive images.

Project extensions

1. Add a table showing your education.
2. Add a hobbies section.
3. Add links to your GitHub profile.
4. Add a favicon.
5. Create separate About and Contact pages.
6. Add a navigation bar.

---

18. HTML Favicon

A favicon is a small icon associated with a website. Browsers commonly show it in the tab, bookmarks, and other browser interfaces.

Example

Place `favicon.ico` in your website directory.

```
website/
|
+-- index.html
+-- favicon.ico
```

Inside the `<head>` element:

```
<link rel="icon" href="favicon.ico">
```

Using a PNG favicon

```
<link rel="icon"
      type="image/png"
      href="images/favicon.png">
```

Using an SVG favicon

```
<link rel="icon"
      type="image/svg+xml"
      href="favicon.svg">
```

Complete example

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">

    <title>My Website</title>

    <link rel="icon" href="favicon.ico">
</head>
<body>

    <h1>Welcome to My Website</h1>

</body>
</html>
```

Important points

- The favicon link belongs inside `<head>`.
- The specified file must exist at the given path.
- Browsers may cache favicons, so changes may not appear immediately.
- A favicon is different from the page title.

---

19. HTML Page Title

The `<title>` element defines the document's title.

It belongs inside `<head>`.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>HTML Learning Notes</title>
</head>
<body>

    <h1>Welcome to HTML</h1>

</body>
</html>
```

The title commonly appears in:

- The browser tab.
- Browser bookmarks.
- Search results, where search engines may choose to display it.

Title versus heading

```
<head>
    <title>My Portfolio Website</title>
</head>

<body>
    <h1>Welcome to My Portfolio</h1>
</body>
```

`<title>`	`<h1>`
Defines the document title.	Defines a heading in the page content.
Located inside `<head>`.	Located inside the page content, typically `<body>`.
Commonly shown in the browser tab.	Displayed within the web page.

Best practices

- Give every page a meaningful title.
- Keep the title concise and descriptive.
- Make titles distinct across different pages.

- Do not confuse the page title with the main heading.

---

20. HTML Tables

HTML tables organize related information into rows and columns.

Important table elements

Element	Purpose
`<table>`	Defines the table.
`<tr>`	Defines a table row.
`<th>`	Defines a header cell.
`<td>`	Defines a data cell.
`<caption>`	Provides a table caption.
`<thead>`	Groups header rows.
`<tbody>`	Groups the main data rows.
`<tfoot>`	Groups footer rows.

Basic table

```
<table>
    <tr>
        <th>Name</th>
        <th>Age</th>
        <th>City</th>
    </tr>

    <tr>
        <td>Aarav</td>
        <td>21</td>
        <td>Mumbai</td>
    </tr>

    <tr>
        <td>Ananya</td>
        <td>22</td>
        <td>Pune</td>
    </tr>
</table>
```

Add CSS borders

```
<style>
    table {
        width: 100%;
        border-collapse: collapse;
    }

    th, td {
        border: 1px solid black;
        padding: 10px;
        text-align: left;
    }

    th {
        background-color: lightgray;
    }
</style>
```

Complete table example

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Student Records</title>

    <style>
        table {
            width: 80%;
            border-collapse: collapse;
        }

        th, td {
            border: 1px solid black;
            padding: 10px;
        }

        th {
            background-color: #dceeff;
        }
    </style>
</head>
<body>

    <h1>Student Records</h1>

    <table>
        <caption>Engineering Students</caption>

        <thead>
            <tr>
                <th scope="col">Roll No.</th>
                <th scope="col">Name</th>
                <th scope="col">Marks</th>
            </tr>
        </thead>

        <tbody>
            <tr>
                <td>101</td>
                <td>Aarav</td>
                <td>85</td>
            </tr>

            <tr>
```

```
                <td>102</td>
                <td>Ananya</td>
                <td>92</td>
            </tr>

            <tr>
                <td>103</td>
                <td>Rohan</td>
                <td>78</td>
            </tr>
        </tbody>
    </table>

</body>
</html>
```

colspan

`colspan` allows a cell to span multiple columns.

```
<table border="1">
    <tr>
        <th colspan="2">Student Information</th>
    </tr>

    <tr>
        <td>Name</td>
        <td>Aarav</td>
    </tr>
</table>
```

The header occupies two columns.

rowspan

`rowspan` allows a cell to span multiple rows.

```
<table border="1">
    <tr>
        <th rowspan="2">Department</th>
        <th>Name</th>
    </tr>

    <tr>
        <td>Computer Science</td>
    </tr>
</table>
```

The first cell occupies two rows.

Important points

- Use tables for tabular data, not general page layout.
- Use `<th>` for headers and `<td>` for ordinary data.
- Use `<caption>` to describe the table.
- Use CSS rather than obsolete presentational attributes for styling.

---

21. HTML Lists

HTML provides three common types of lists.

1. Ordered lists.
2. Unordered lists.
3. Description lists.

21.1 Ordered list

An ordered list uses `<ol>` and displays items in a sequence.

Each item is written inside `<li>`.

```
<ol>
    <li>Learn HTML</li>
    <li>Learn CSS</li>
    <li>Learn JavaScript</li>
</ol>
```

Typical output:

1. Learn HTML

2. Learn CSS
3. Learn JavaScript

Start numbering from a particular value

```
<ol start="5">
    <li>HTML</li>
    <li>CSS</li>
    <li>JavaScript</li>
</ol>
```

The numbering starts at 5.

Different numbering styles

```
<ol type="A">
    <li>HTML</li>
    <li>CSS</li>
</ol>
```

Common `type` values:

- `1` — numbers.
- `A` — uppercase letters.
- `a` — lowercase letters.
- `I` — uppercase Roman numerals.
- `i` — lowercase Roman numerals.

21.2 Unordered list

An unordered list uses `<ul>`.

```
<ul>
    <li>Apple</li>
    <li>Mango</li>
    <li>Orange</li>
</ul>
```

Typical output:

- Apple
- Mango
- Orange

CSS can customize list markers.

```
<style>
    .square-list {
        list-style-type: square;
    }
</style>

<ul class="square-list">
    <li>Java</li>
    <li>Python</li>
    <li>C++</li>
</ul>
```

21.3 Description list

A description list uses:

- `<dl>` for the list.
- `<dt>` for each term.
- `<dd>` for the term's description.

```
<dl>
    <dt>HTML</dt>
    <dd>Structures web page content.</dd>

    <dt>CSS</dt>
    <dd>Controls the presentation of web pages.</dd>

    <dt>JavaScript</dt>
    <dd>Adds behavior and interactivity.</dd>
</dl>
```

21.4 Nested lists

A list can contain another list.

```
<ul>
    <li>Frontend Development
        <ul>
            <li>HTML</li>
            <li>CSS</li>
            <li>JavaScript</li>
        </ul>
    </li>

    <li>Backend Development
        <ul>
            <li>Java</li>
            <li>Python</li>
        </ul>
    </li>
</ul>
```

21.5 Complete list example

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>HTML Lists</title>
</head>
<body>

    <h1>My Learning Plan</h1>

    <h2>Topics in Order</h2>
    <ol>
        <li>HTML basics</li>
        <li>HTML elements</li>
        <li>CSS basics</li>
        <li>JavaScript basics</li>
    </ol>

    <h2>My Skills</h2>
    <ul>
        <li>SQL</li>
        <li>Java</li>
        <li>C++</li>
    </ul>

    <h2>Web Technologies</h2>
    <dl>
        <dt>HTML</dt>
        <dd>Defines content structure.</dd>

        <dt>CSS</dt>
        <dd>Defines presentation.</dd>

        <dt>JavaScript</dt>
        <dd>Defines interactive behavior.</dd>
    </dl>

</body>
</html>
```

---

22. Practice Questions

Beginner Level

Question 1

Create a web page containing a heading and two paragraphs.

Solution:

```
<!DOCTYPE html>
<html>
<head>
    <title>My First Page</title>
</head>
<body>
    <h1>My Introduction</h1>
    <p>My name is Aarav.</p>
    <p>I am learning HTML.</p>
</body>
</html>
```

Question 2

Create a paragraph with bold, emphasized, and highlighted text.

Solution:

```
<p>
    This is <strong>important</strong>,
    this is <em>emphasized</em>,
    and this is <mark>highlighted</mark>.
</p>
```

Question 3

Create a link to Wikipedia.

Solution:

```
<a href="https://www.wikipedia.org/">
    Visit Wikipedia
</a>
```

Question 4

Display an image with alternative text.

Solution:

```
<img src="photo.jpg"
     alt="A photograph"
     width="300">
```

Ensure that `photo.jpg` exists at the specified path.

Intermediate Level

Question 5

Create an ordered list of five subjects.

Solution:

```
<ol>
    <li>Mathematics</li>
    <li>Physics</li>
    <li>Computer Science</li>
    <li>English</li>
    <li>Database Systems</li>
</ol>
```

Question 6

Create a student table with three columns and two student records.

Solution:

```
<table border="1">
    <tr>
        <th>Roll No.</th>
        <th>Name</th>
        <th>Marks</th>
    </tr>

    <tr>
        <td>1</td>
        <td>Aarav</td>
        <td>85</td>
    </tr>

    <tr>
        <td>2</td>
        <td>Ananya</td>
        <td>90</td>
    </tr>
</table>
```

Note: `border="1"` is used here for a quick demonstration. CSS is preferred for modern styling.

Question 7

Create an HTML page using an external stylesheet.

Solution:

`index.html`

```
<!DOCTYPE html>
<html>
<head>
    <title>External CSS Practice</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>My Website</h1>
    <p>Learning external CSS.</p>
</body>
</html>
```

`style.css`

```
body {
    font-family: Arial, sans-serif;
    background-color: lightgray;
}

h1 {
    color: navy;
}

p {
    color: darkgreen;
}
```

Question 8

Create a page with links that navigate to three sections.

Solution:

```
<!DOCTYPE html>
<html>
<head>
    <title>Page Navigation</title>
</head>
<body>

    <nav>
        <a href="#about">About</a>
        <a href="#skills">Skills</a>
        <a href="#contact">Contact</a>
    </nav>

    <section id="about">
        <h2>About</h2>
        <p>About me.</p>
    </section>

    <section id="skills">
        <h2>Skills</h2>
        <p>My technical skills.</p>
    </section>

    <section id="contact">
        <h2>Contact</h2>
        <p>My contact information.</p>
    </section>

</body>
</html>
```

Mini Project Challenge

Create a personal portfolio page containing:

- A meaningful page title and favicon.
- One main heading.
- An introduction paragraph.
- A profile image with appropriate alternative text.
- A list of skills.
- A table of education details.
- Links to your projects.
- An email link.

- External CSS.

Try completing the project without looking at the earlier examples first.

---

23. Important HTML Viva Questions

1. What is HTML?

HTML is a markup language used to structure content on web pages.

2. What is the full form of HTML?

HyperText Markup Language.

3. Is HTML a programming language?

No. HTML is a markup language because it describes document structure rather than general-purpose computational logic.

4. What is an HTML tag?

A tag is markup written inside angle brackets, such as `<p>` or `</p>`.

5. What is an HTML element?

An element consists of its tag or tags and associated content, where applicable.

6. What is an attribute?

An attribute provides additional information or configuration for an HTML element.

7. What is the purpose of `<!DOCTYPE html>`?

It declares the document type and helps the browser use standards mode.

8. What is the difference between `<head>` and `<body>`?

`<head>` contains document metadata and linked resources. `<body>` contains the page content.

9. What is the difference between `<title>` and `<h1>`?

`<title>` defines the document title, while `<h1>` defines a heading displayed within the page.

10. What is the difference between `<b>` and `<strong>`?

`<b>` draws attention without implying strong importance. `<strong>` indicates strong importance.

11. What is the difference between `<ol>` and `<ul>`?

`<ol>` represents an ordered list. `<ul>` represents an unordered list.

12. What is the purpose of the `alt` attribute?

It provides alternative text for an image.

13. What is a hyperlink?

A hyperlink is a link to another resource or location, commonly created with `<a>`.

14. What is a favicon?

A small icon associated with a website, commonly displayed in browser tabs.

15. What is CSS?

CSS stands for Cascading Style Sheets and controls the presentation and layout of web pages.

16. What are the three ways to add CSS?

Inline CSS, internal CSS, and external CSS.

17. What is the difference between `id` and `class`?

An `id` identifies a particular element within a document, while a `class` can be shared by multiple elements.

18. What is the purpose of `<br>`?

It inserts a line break.

19. What is the purpose of `<hr>`?

It represents a thematic break between sections.

20. What is the purpose of `<table>`?

It represents tabular data arranged in rows and columns.

21. What is the difference between `<th>` and `<td>`?

`<th>` represents a header cell, while `<td>` represents a data cell.

22. What is the difference between `colspan` and `rowspan`?

`colspan` makes a cell span multiple columns. `rowspan` makes a cell span multiple rows.

23. What are empty elements?

Elements that do not have content or closing tags, such as `<br>`, `<hr>`, and `<img>`.

24. What is a nested element?

An element placed inside another element.

25. Why should we use semantic HTML?

Semantic HTML communicates the meaning and structure of content, helping accessibility, maintainability, and search engines understand the page.

Final Revision Checklist

Before moving to advanced HTML, make sure you can:

- [ ] Write a complete HTML document from memory.
- [ ] Explain tags, elements, and attributes.
- [ ] Use all six heading levels correctly.
- [ ] Create paragraphs and line breaks.
- [ ] Format text with meaningful HTML elements.
- [ ] Add comments without exposing sensitive information.
- [ ] Apply colors and inline CSS.
- [ ] Connect an external CSS file.
- [ ] Create absolute, relative, email, and section links.
- [ ] Insert images using correct paths and alternative text.
- [ ] Add a favicon and document title.
- [ ] Create tables with headers, captions, `colspan`, and `rowspan`.
- [ ] Create ordered, unordered, and description lists.
- [ ] Build a complete personal profile website.

HTML Complete Notes — Part 3: HTML Forms

Table of Contents

42. HTML Forms
43. HTML Form Attributes
44. HTML Form Elements
45. HTML Input Types
46. HTML Input Attributes
47. Input Form Attributes
48. Complete HTML Form Project
49. Practice Questions with Solutions
50. Important Viva Questions

---

42. HTML Forms

42.1 What is an HTML form?

An HTML form is a section of a web page that collects information from users.

Forms allow users to enter data, select options, upload files, and submit information.

Examples of forms include:

- Login forms
- Registration forms
- Contact forms
- Feedback forms
- Search forms
- Online application forms
- Payment forms

The `<form>` element is used to create a form.

42.2 Basic syntax

```
<form action="/submit" method="post">
    <!-- Form controls go here -->
</form>
```

Explanation

- `<form>` defines the form.
- `action` specifies the URL that will process the submitted data.
- `method` specifies how the browser sends the form data.
- The content inside `<form>` contains controls such as text fields, radio buttons, and submit buttons.

The URL in this example is illustrative. A working submission requires a server endpoint that accepts and processes the request.

42.3 Your first HTML form

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>My First Form</title>
</head>
<body>

    <h1>Student Registration</h1>

    <form action="/register" method="post">

        <label for="fullname">Full Name:</label>
        <input type="text" id="fullname" name="fullname">

        <br><br>

        <label for="email">Email:</label>
        <input type="email" id="email" name="email">

        <br><br>

        <button type="submit">Register</button>

    </form>

</body>
</html>
```

Understanding the form step by step

1. The user opens the registration page.
2. The user enters a name and email address.
3. The browser performs applicable client-side validation.
4. When the user clicks Register, the browser prepares the successful form controls for submission.
5. The browser sends the data to `/register` using the POST method.
6. The server must receive and process the request.

Important: HTML creates the form interface, but it does not automatically create a database record. Saving data requires a backend or another appropriate service.

42.4 What is a label?

A `<label>` describes the purpose of a form control.

```
<label for="username">Username:</label>
<input type="text" id="username" name="username">
```

The `for` value must match the control's `id`.

Benefits:

- Makes the form easier to understand.
- Helps screen-reader users identify controls.
- Clicking the label can focus or activate its associated control.

42.5 What is the purpose of the `name` attribute?

The `name` attribute identifies the field when the form data is submitted.

```
<input type="text" name="username">
```

If the user enters `Prajwal`, the submitted data can contain:

```
username=Prajwal
```

A control generally needs a `name` to contribute a field to normal form submission.

The `id` and `name` attributes are different:

Attribute	Purpose
`id`	Identifies the element in the document.
`name`	Identifies the submitted field.

---

43. HTML Form Attributes

Form attributes control where and how the browser submits form data.

43.1 The `action` attribute

The `action` attribute specifies the URL that receives the submitted form data.

```
<form action="/register" method="post">
    <input type="text" name="username">
    <button type="submit">Register</button>
</form>
```

When submitted, the browser sends the form data to `/register`.

If `action` is omitted, the form generally submits to the current document URL.

43.2 The `method` attribute

The `method` attribute specifies the HTTP method used to submit the form.

The two most commonly used values are `get` and `post`.

GET method

```
<form action="/search" method="get">
    <label for="query">Search:</label>
    <input type="search" id="query" name="q">

    <button type="submit">Search</button>
</form>
```

If the user searches for HTML, the URL may look like:

```
/search?q=HTML
```

Characteristics:

- Form data is generally included in the URL query string.
- Useful for searches and other read-only requests.
- URLs can be bookmarked and shared.
- Not suitable for secrets because query values can appear in browser history, logs, and URLs.

POST method

```
<form action="/register" method="post">
    <label for="username">Username:</label>
    <input type="text" id="username" name="username">

    <button type="submit">Register</button>
</form>
```

Characteristics:

- Form data is sent in the request body for a typical POST form submission.
- Commonly used to create or update server-side data.
- Data is not ordinarily displayed in the URL query string.
- POST does not automatically make data secure. HTTPS and appropriate server-side security are still necessary.

GET vs. POST

GET	POST
Commonly used for searching and retrieving information.	Commonly used for submitting data that may change server state.
Data is usually included in the URL query string.	Data is usually included in the request body.
Requests can often be bookmarked and shared.	Request bodies are not normally preserved as part of a bookmark.
Not suitable for confidential information.	Does not itself provide confidentiality.
Usually used for read-only operations.	Often used for operations that create or modify data.

43.3 The `enctype` attribute

`enctype` specifies how the form data is encoded when using POST.

Default encoding

```
<form action="/submit" method="post">
    <input type="text" name="username">
    <button type="submit">Submit</button>
</form>
```

The default is:

`application/x-www-form-urlencoded`

Multipart encoding

For file uploads, use:

```
<form action="/upload"
      method="post"
      enctype="multipart/form-data">

    <label for="document">Choose a document:</label>
    <input type="file" id="document" name="document">

    <button type="submit">Upload</button>
</form>
```

The server must support file uploads and process the multipart request.

Plain-text encoding

`text/plain` is another possible encoding, but it is rarely appropriate for production forms.

43.4 The `target` attribute

The `target` attribute specifies where the response should be displayed.

```
<form action="/search" method="get" target="_blank">
    <input type="search" name="q">
    <button type="submit">Search</button>
</form>
```

Common values include:

- `_self`: Current browsing context.
- `_blank`: A new browsing context.
- `_parent`: Parent browsing context.
- `_top`: Top-level browsing context.

43.5 The `autocomplete` attribute

This attribute controls whether the browser may offer autocomplete suggestions for form fields.

```
<form autocomplete="on">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name">
</form>
```

You can also use it on individual controls:

```
<input type="email"
       name="email"
       autocomplete="email">
```

Common values include `on`, `off`, `name`, `email`, `username`, and `current-password`.

43.6 The `novalidate` attribute

This attribute disables the browser's built-in constraint validation when submitting the form.

```
<form action="/submit" method="post" novalidate>
    <input type="email" name="email" required>
    <button type="submit">Submit</button>
</form>
```

Even though the field is marked `required`, the browser will not block submission because of that constraint when the form is submitted normally.

The server should still validate all received data.

43.7 The `accept-charset` attribute

This attribute specifies the character encoding used for form submission.

```
<form action="/submit" method="post" accept-charset="UTF-8">
    <input type="text" name="name">
    <button type="submit">Submit</button>
</form>
```

UTF-8 is the standard practical choice for modern web forms.

43.8 Complete example using form attributes

```
<form action="/register"
      method="post"
      autocomplete="on"
      accept-charset="UTF-8">

    <label for="fullname">Full Name:</label>
    <input type="text"
           id="fullname"
           name="fullname"
           autocomplete="name"
           required>

    <button type="submit">Register</button>
</form>
```

---

44. HTML Form Elements

Form elements are the controls and supporting elements used to collect user input.

44.1 The `<input>` element

The `<input>` element creates many types of controls.

```
<input type="text" name="username">
```

The `type` attribute determines the kind of input.

44.2 The `<label>` element

```
<label for="age">Age:</label>
<input type="number" id="age" name="age">
```

The label explains what the control is for.

44.3 The `<textarea>` element

A `<textarea>` allows users to enter multiline text.

```
<label for="message">Message:</label>

<textarea id="message"
          name="message"
          rows="5"
          cols="30"></textarea>
```

Unlike an ordinary text input, a textarea supports multiple lines.

44.4 The `<select>` element

A `<select>` creates a dropdown menu.

```
<label for="city">Choose a city:</label>

<select id="city" name="city">
    <option value="mumbai">Mumbai</option>
    <option value="pune">Pune</option>
    <option value="nashik">Nashik</option>
</select>
```

The submitted value comes from the selected option's `value` attribute.

44.5 The `<option>` element

An `<option>` represents one selectable item in a dropdown or list.

```
<select name="course">
    <option value="">Select a course</option>
    <option value="computer">Computer Engineering</option>
    <option value="mechanical">Mechanical Engineering</option>
</select>
```

44.6 The `<optgroup>` element

Use `<optgroup>` to group related dropdown options.

```
<label for="course">Course:</label>

<select id="course" name="course">
    <optgroup label="Engineering">
        <option value="computer">Computer Engineering</option>
        <option value="mechanical">Mechanical Engineering</option>
    </optgroup>

    <optgroup label="Science">
        <option value="physics">Physics</option>
        <option value="chemistry">Chemistry</option>
    </optgroup>
</select>
```

44.7 The `<button>` element

```
<button type="submit">Submit Form</button>
```

A button can submit a form, reset controls, or perform a custom action.

44.8 The `<fieldset>` element

A `<fieldset>` groups related form controls.

```
<fieldset>
    <legend>Personal Information</legend>

    <label for="fname">First Name:</label>
    <input type="text" id="fname" name="fname">

    <label for="lname">Last Name:</label>
    <input type="text" id="lname" name="lname">
</fieldset>
```

44.9 The `<legend>` element

The `<legend>` provides a caption for a `<fieldset>`.

```
<fieldset>
    <legend>Account Details</legend>
    <input type="text" name="username">
</fieldset>
```

44.10 The `<datalist>` element

A `<datalist>` provides suggestions for an input. Users can generally enter other values too.

```
<label for="browser">Choose a browser:</label>

<input list="browsers" id="browser" name="browser">

<datalist id="browsers">
    <option value="Chrome">
    <option value="Firefox">
    <option value="Edge">
    <option value="Safari">
</datalist>
```

Unlike a `<select>`, a datalist does not restrict the user to its listed suggestions.

44.11 The `<output>` element

The `<output>` element represents the result of a calculation or user action.

```
<form oninput="result.value = Number(a.value) + Number(b.value)">
    <input type="number" id="a" name="a" value="0">
    +
    <input type="number" id="b" name="b" value="0">
    =
    <output name="result" for="a b">0</output>
</form>
```

Try changing the numbers. The result updates automatically.

Summary of form elements

Element	Purpose
`<form>`	Contains the form controls.
`<input>`	Creates different input controls.
`<label>`	Describes a control.
`<textarea>`	Collects multiline text.
`<select>`	Creates a dropdown or selection list.
`<option>`	Defines a selectable option.

Element	Purpose
`<optgroup>`	Groups related options.
`<button>`	Creates a button.
`<fieldset>`	Groups related controls.
`<legend>`	Labels a fieldset.
`<datalist>`	Provides input suggestions.
`<output>`	Represents a result.

---

45. HTML Input Types

The `type` attribute determines how an `<input>` behaves and appears.

45.1 Text input

```
<label for="username">Username:</label>
<input type="text" id="username" name="username">
```

Used for a single line of ordinary text.

45.2 Password input

```
<label for="password">Password:</label>
<input type="password" id="password" name="password">
```

Characters are visually obscured, but this does not encrypt the data or provide authentication by itself.

45.3 Email input

```
<label for="email">Email:</label>
<input type="email" id="email" name="email" required>
```

Provides email-oriented input behavior and basic browser validation.

It does not prove that an email address exists.

45.4 Number input

```
<label for="age">Age:</label>
<input type="number"
       id="age"
       name="age"
       min="1"
       max="120">
```

Useful for numeric values.

45.5 Radio buttons

Radio buttons let users select one option from a group.

```
<p>Gender:</p>

<label>
    <input type="radio" name="gender" value="male">
    Male
</label>

<label>
    <input type="radio" name="gender" value="female">
    Female
</label>

<label>
    <input type="radio" name="gender" value="other">
    Other
</label>
```

Radio buttons belong to the same group when they share the same `name`.

Only the selected radio button in a group is normally submitted.

45.6 Checkboxes

Checkboxes let users independently select multiple options.

```
<p>Select your skills:</p>

<label>
    <input type="checkbox" name="skills" value="html">
    HTML
</label>

<label>
    <input type="checkbox" name="skills" value="css">
    CSS
</label>

<label>
    <input type="checkbox" name="skills" value="javascript">
    JavaScript
</label>
```

If several boxes are checked, multiple values can be submitted under the same name.

45.7 Date input

```
<label for="dob">Date of Birth:</label>
<input type="date" id="dob" name="dob">
```

The browser typically displays a date picker.

45.8 Time input

```
<label for="meeting">Meeting Time:</label>
<input type="time" id="meeting" name="meeting">
```

45.9 Datetime-local input

```
<label for="appointment">Appointment:</label>
<input type="datetime-local"
       id="appointment"
       name="appointment">
```

Collects a local date and time without specifying a timezone.

45.10 Month input

```
<label for="month">Choose a month:</label>
<input type="month" id="month" name="month">
```

45.11 Week input

```
<label for="week">Choose a week:</label>
<input type="week" id="week" name="week">
```

45.12 Color input

```
<label for="color">Choose a color:</label>
<input type="color" id="color" name="color" value="#0000ff">
```

45.13 Range input

```
<label for="volume">Volume:</label>
<input type="range"
       id="volume"
       name="volume"
       min="0"
       max="100"
       value="50">
```

Useful for selecting a value within a range.

45.14 File input

```
<label for="resume">Upload Resume:</label>
<input type="file" id="resume" name="resume">
```

To allow multiple files:

```
<input type="file" name="documents" multiple>
```

To suggest PDF files:

```
<input type="file"
       name="resume"
       accept=".pdf,application/pdf">
```

The `accept` attribute is a hint to the file picker, not a security guarantee. The server must validate uploaded files.

45.15 Hidden input

```
<input type="hidden" name="form_version" value="registration-v1">
```

Hidden inputs are not displayed like normal controls, but their values may be submitted.

Important: Hidden inputs are not secret. Users can inspect and modify them.

45.16 Search input

```
<label for="search">Search:</label>
<input type="search" id="search" name="q">
```

45.17 Telephone input

```
<label for="phone">Phone Number:</label>
<input type="tel" id="phone" name="phone">
```

This can provide a telephone-friendly keyboard on mobile devices. It does not enforce a universal phone number format automatically.

45.18 URL input

```
<label for="website">Website:</label>
<input type="url" id="website" name="website">
```

Provides basic browser validation for URL-like values.

45.19 Submit input

```
<input type="submit" value="Submit Form">
```

Submits the form when activated.

45.20 Reset input

```
<input type="reset" value="Reset Form">
```

Resets form controls to their initial values. Use reset buttons carefully because users may accidentally discard their input.

45.21 Button input

```
<input type="button" value="Click Me">
```

This creates a general-purpose button. It needs JavaScript or another relevant behavior to perform a custom action.

45.22 Image input

```
<input type="image"
       src="submit.png"
       alt="Submit form"
       width="100">
```

An image input acts as a graphical submit button.

Summary of input types

Type	Purpose
`text`	Single-line text.
`password`	Password-style input.
`email`	Email address.
`number`	Numeric value.
`radio`	One selection from a group.
`checkbox`	Independent selections.
`date`	Calendar date.
`time`	Time of day.
`datetime-local`	Local date and time.
`month`	Month and year.
`week`	Week and year.
`color`	Color selection.

Type	Purpose
`range`	Value within a range.
`file`	File selection.
`hidden`	Hidden submitted value.
`search`	Search text.
`tel`	Telephone-oriented input.
`url`	URL-oriented input.
`submit`	Submit button.
`reset`	Reset button.
`button`	General-purpose button.
`image`	Graphical submit button.

---

46. HTML Input Attributes

Input attributes configure the behavior, validation, and initial values of form controls.

46.1 The `value` attribute

Specifies an initial value for many input types.

```
<input type="text" name="username" value="Prajwal">
```

The field initially displays `Prajwal`.

For checkboxes and radio buttons, `value` specifies the value submitted when the control is selected.

```
<input type="radio" name="course" value="html">
```

46.2 The `placeholder` attribute

Displays a short hint when the field is empty.

```
<input type="text"
       name="username"
       placeholder="Enter your username">
```

A placeholder is not a substitute for a visible label.

46.3 The `required` attribute

Makes a control required for normal browser form validation.

```
<input type="email" name="email" required>
```

The browser can prevent submission when the required field is empty.

The server must still validate submitted data.

46.4 The `readonly` attribute

Makes a supported control non-editable by the user.

```
<input type="text"
       name="student_id"
       value="STU101"
       readonly>
```

A read-only control can still be submitted if it has a name.

46.5 The `disabled` attribute

Disables a control.

```
<input type="text" name="username" disabled>
```

A disabled control is generally not focusable, does not participate in ordinary constraint validation, and is not included in form submission.

Readonly vs. disabled

Readonly	Disabled
Cannot normally be edited by the user.	Cannot normally be interacted with.
Can generally receive focus.	Normally cannot receive focus.
Can be submitted if it has a name.	Is not submitted as a successful control.

Readonly	Disabled
Available only on supported control types.	Available on many form controls.

46.6 The `maxlength` attribute

Sets the maximum number of characters for supported text inputs.

```
<input type="text"
       name="username"
       maxlength="20">
```

46.7 The `minlength` attribute

Sets the minimum number of characters for supported text inputs.

```
<input type="text"
       name="username"
       minlength="3"
       maxlength="20">
```

46.8 The `min` and `max` attributes

Set the allowed minimum and maximum for supported numeric and date-related inputs.

```
<input type="number"
       name="age"
       min="18"
       max="60">
```

The browser can reject values outside the specified range during normal constraint validation.

46.9 The `step` attribute

Defines the permitted increments for supported numeric and date/time inputs.

```
<input type="number"
       name="price"
       min="0"
       step="0.01">
```

This allows values such as `10.25` and `10.50`.

For a number input, the default step is generally `1`, unless another rule applies.

46.10 The `pattern` attribute

Defines a regular expression used to validate supported text-like input types.

```
<label for="code">Student Code:</label>

<input type="text"
       id="code"
       name="code"
       pattern="[A-Z]{2}[0-9]{4}"
       title="Enter two uppercase letters followed by four digits"
       required>
```

Example accepted value: `AB1234`.

The `pattern` attribute is not supported as a general validation mechanism for every input type. Use it only with supported text-like types.

46.11 The `checked` attribute

Initially checks a checkbox or radio button.

```
<input type="checkbox" name="subscribe" value="yes" checked>
```

46.12 The `multiple` attribute

Allows multiple values for supported input types, such as email and file.

```
<input type="file" name="files" multiple>
```

Another example:

```
<input type="email" name="emails" multiple>
```

46.13 The `accept` attribute

Suggests which file types should be selectable in a file picker.

```
<input type="file"
       name="photo"
       accept="image/*">
```

This does not guarantee that the uploaded file is safe or genuinely of that type.

46.14 The `autocomplete` attribute

Provides autocomplete hints.

```
<input type="text"
       name="fullname"
       autocomplete="name">

<input type="email"
       name="email"
       autocomplete="email">

<input type="password"
       name="password"
       autocomplete="new-password">
```

46.15 The `autofocus` attribute

Requests that a control receive focus when the page loads.

```
<input type="text" name="username" autofocus>
```

Use it sparingly, as automatic focus can be disruptive for some users.

46.16 The `size` attribute

Specifies the visible width of supported text-like input fields in approximate character units.

```
<input type="text" name="username" size="30">
```

CSS is generally preferable for controlling layout.

46.17 The `list` attribute

Connects an input to a `<datalist>`.

```
<label for="browser">Browser:</label>

<input type="text" id="browser" name="browser" list="browser-options">

<datalist id="browser-options">
    <option value="Chrome">
    <option value="Firefox">
    <option value="Edge">
</datalist>
```

46.18 The `inputmode` attribute

Suggests an appropriate virtual keyboard on devices that support it.

```
<input type="text" name="pin" inputmode="numeric">
```

This can suggest a numeric keyboard, but does not validate the input by itself.

46.19 The `enterkeyhint` attribute

Hints at the label or purpose of the Enter key on virtual keyboards.

```
<input type="text"
       name="search"
       enterkeyhint="search">
```

Input attributes summary

Attribute	Purpose
`value`	Initial or submitted value, depending on input type.
`placeholder`	Displays a short hint.
`required`	Requires a value for normal validation.
`readonly`	Prevents normal user editing.
`disabled`	Disables the control.
`maxlength`	Sets maximum text length.
`minlength`	Sets minimum text length.
`min`	Sets a minimum allowed value.

Attribute	Purpose
`max`	Sets a maximum allowed value.
`step`	Sets allowed increments.
`pattern`	Defines a validation pattern for supported types.
`checked`	Initially checks a checkbox or radio.
`multiple`	Allows multiple values for supported types.
`accept`	Suggests file types.
`autocomplete`	Provides autocomplete instructions.
`autofocus`	Requests initial focus.
`size`	Sets approximate visible input width.
`list`	Connects to a datalist.
`inputmode`	Suggests a virtual keyboard.
`enterkeyhint`	Hints at the virtual Enter key action.

---

47. Input Form Attributes

Input form attributes are attributes that allow an individual form control to specify submission behavior or associate with a form.

These are different from the general input attributes covered in the previous section.

47.1 The `form` attribute

The `form` attribute associates a form control with a form using the form's ID.

The control does not have to be physically inside the `<form>` element.

```
<!DOCTYPE html>
<html>
<head>
    <title>Form Attribute</title>
</head>
<body>

    <form id="student-form" action="/submit" method="post">
        <label for="name">Name:</label>
        <input type="text" id="name" name="name">
    </form>

    <label for="email">Email:</label>
    <input type="email"
           id="email"
           name="email"
           form="student-form">

    <button type="submit" form="student-form">
        Submit
    </button>

</body>
</html>
```

The email field and submit button are associated with `student-form`.

This is useful when a page layout requires controls to appear outside the form's immediate markup.

47.2 The `formaction` attribute

Overrides the form's `action` URL for a particular submit button or image submit control.

```
<form action="/save-draft" method="post">
    <input type="text" name="title">

    <button type="submit">Save Draft</button>

    <button type="submit"
            formaction="/publish">
        Publish
    </button>
</form>
```

The first button uses `/save-draft`. The second uses `/publish`.

The server must implement both endpoints.

47.3 The `formenctype` attribute

Overrides the form's encoding type for a particular submit button.

```
<form action="/submit"
      method="post"
      enctype="application/x-www-form-urlencoded">

    <input type="text" name="username">

    <input type="file" name="document">

    <button type="submit"
            formenctype="multipart/form-data">
        Upload
    </button>
</form>
```

The button requests multipart encoding for that submission.

47.4 The `formmethod` attribute

Overrides the form's method for a particular submit button.

```
<form action="/search" method="get">
    <input type="search" name="q">

    <button type="submit">Search with GET</button>

    <button type="submit"
            formmethod="post">
        Submit with POST
    </button>
</form>
```

The second button requests POST rather than GET.

47.5 The `formnovalidate` attribute

Allows a particular submit button to bypass the browser's built-in constraint validation.

```
<form action="/register" method="post">
    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>

    <button type="submit">Register</button>

    <button type="submit" formnovalidate>
        Submit Without Browser Validation
    </button>
</form>
```

The second button bypasses client-side constraint validation for that submission.

The server should still validate the data.

47.6 The `formtarget` attribute

Overrides the form's target for a particular submit button.

```
<form action="/search" method="get">
    <input type="search" name="q">

    <button type="submit">Open Here</button>

    <button type="submit" formtarget="_blank">
        Open Response in New Tab
    </button>
</form>
```

The second button requests that the response be opened in a new browsing context.

Summary of input form attributes

Attribute	Purpose
`form`	Associates a control with a form by ID.
`formaction`	Overrides the submission URL.
`formenctype`	Overrides the form data encoding.
`formmethod`	Overrides the submission method.
`formnovalidate`	Bypasses browser constraint validation for that submission.
`formtarget`	Overrides where the response is displayed.

Important distinction

- `action`, `method`, and `enctype` are normally form attributes.
- `formaction`, `formmethod`, and `formenctype` are submit-control attributes.
- The corresponding `form*` attributes override the general form settings for a particular submission.

---

48. Complete HTML Form Project

Project: Student Registration Form

This project combines form elements, input types, validation attributes, dropdowns, radio buttons, checkboxes, and file uploads.

Complete HTML program

Save this file as `registration.html`.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">

    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">

    <title>Student Registration Form</title>

    <style>
        * {
            box-sizing: border-box;
        }

        body {
            font-family: Arial, sans-serif;
            background-color: #f2f5f9;
            margin: 0;
            padding: 20px;
        }

        .container {
            max-width: 650px;
            margin: 20px auto;
            padding: 25px;
            background-color: white;
            border-radius: 10px;
        }

        h1 {
            text-align: center;
            color: navy;
        }

        label {
            display: block;
            margin-top: 14px;
            margin-bottom: 6px;
        }

        input:not([type="radio"]):not([type="checkbox"]),
        select,
        textarea {
            width: 100%;
            padding: 10px;
            border: 1px solid #cccccc;
```

```
            border-radius: 5px;
        }

        .choice-label {
            display: inline-block;
            margin-right: 12px;
        }

        fieldset {
            margin-top: 18px;
            border: 1px solid #cccccc;
            border-radius: 5px;
        }

        button {
            margin-top: 20px;
            padding: 12px 18px;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }

        .submit-btn {
            background-color: navy;
            color: white;
        }

        .reset-btn {
            background-color: #dddddd;
        }
    </style>
</head>

<body>

    <div class="container">

        <h1>Student Registration</h1>

        <form action="/register"
              method="post"
              autocomplete="on">

            <label for="fullname">Full Name:</label>

            <input type="text"
                   id="fullname"
```

```
        name="fullname"
        placeholder="Enter your full name"
        autocomplete="name"
        minlength="3"
        maxlength="60"
        required>

<label for="email">Email Address:</label>

<input type="email"
        id="email"
        name="email"
        placeholder="Enter your email"
        autocomplete="email"
        required>

<label for="phone">Phone Number:</label>

<input type="tel"
        id="phone"
        name="phone"
        placeholder="Enter your phone number"
        autocomplete="tel">

<label for="dob">Date of Birth:</label>

<input type="date"
        id="dob"
        name="dob">

<fieldset>
    <legend>Gender</legend>

    <label class="choice-label">
        <input type="radio"
               name="gender"
               value="male">
        Male
    </label>

    <label class="choice-label">
        <input type="radio"
               name="gender"
               value="female">
        Female
    </label>
```

```
        <label class="choice-label">
            <input type="radio"
                   name="gender"
                   value="other">
            Other
        </label>
    </fieldset>

    <label for="course">Select Course:</label>

    <select id="course" name="course" required>
        <option value="">Choose a course</option>
        <option value="computer">Computer Engineering</option>
        <option value="mechanical">Mechanical Engineering</option>
        <option value="civil">Civil Engineering</option>
    </select>

    <fieldset>
        <legend>Technical Skills</legend>

        <label class="choice-label">
            <input type="checkbox"
                   name="skills"
                   value="html">
            HTML
        </label>

        <label class="choice-label">
            <input type="checkbox"
                   name="skills"
                   value="css">
            CSS
        </label>

        <label class="choice-label">
            <input type="checkbox"
                   name="skills"
                   value="javascript">
            JavaScript
        </label>
    </fieldset>

    <label for="resume">Upload Resume (PDF):</label>

    <input type="file"
           id="resume"
           name="resume"
```

```
                accept=".pdf,application/pdf">

            <label for="address">Address:</label>

            <textarea id="address"
                      name="address"
                      rows="4"
                      placeholder="Enter your address"></textarea>

            <label>
                <input type="checkbox"
                       name="terms"
                       value="accepted"
                       required>
                I agree to the terms and conditions.
            </label>

            <button type="submit" class="submit-btn">
                Register
            </button>

            <button type="reset" class="reset-btn">
                Reset
            </button>

        </form>

    </div>

</body>
</html>
```

What does this project demonstrate?

1. Text, email, telephone, and date inputs.
2. Radio buttons and checkboxes.
3. Dropdown menus.
4. File selection.
5. Multiline text input.
6. Labels and fieldsets.
7. Required fields and basic validation.
8. CSS styling and responsive container sizing.

9. Submit and reset buttons.

Important project note

The example uses `action="/register"` as a placeholder backend endpoint. The form interface will render in a browser, but successful server-side registration requires a backend implementation.

For a real registration system, validate the submitted data on the server, safely process uploaded files, and use HTTPS.

---

49. Practice Questions with Solutions

Question 1: Create a login form

Requirements:

- Username
- Password
- Submit button

Solution

```
<form action="/login" method="post">
    <label for="username">Username:</label>
    <input type="text"
           id="username"
           name="username"
           required>

    <label for="password">Password:</label>
    <input type="password"
           id="password"
           name="password"
           required>

    <button type="submit">Login</button>
</form>
```

Question 2: Create a form with a dropdown

Requirements:

- Name

- City dropdown
- Submit button

Solution

```
<form action="/submit" method="post">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" required>

    <label for="city">City:</label>
    <select id="city" name="city" required>
        <option value="">Select City</option>
        <option value="mumbai">Mumbai</option>
        <option value="pune">Pune</option>
        <option value="nashik">Nashik</option>
    </select>

    <button type="submit">Submit</button>
</form>
```

Question 3: Create a form with radio buttons and checkboxes

Solution

```
<form action="/submit" method="post">
    <p>Study Mode:</p>

    <label>
        <input type="radio" name="mode" value="online" required>
        Online
    </label>

    <label>
        <input type="radio" name="mode" value="offline">
        Offline
    </label>

    <p>Interests:</p>

    <label>
        <input type="checkbox" name="interest" value="coding">
        Coding
    </label>

    <label>
        <input type="checkbox" name="interest" value="design">
        Design
    </label>

    <button type="submit">Submit</button>
</form>
```

Question 4: Create a field that accepts only numbers between 18 and 60

Solution

```
<label for="age">Age:</label>

<input type="number"
       id="age"
       name="age"
       min="18"
       max="60"
       required>
```

Question 5: Create a text field with a maximum length of 15 characters

Solution

```
<input type="text"
       name="username"
       maxlength="15">
```

Question 6: Create a field for a student code such as AB1234

Solution

```
<label for="student-code">Student Code:</label>

<input type="text"
       id="student-code"
       name="student_code"
       pattern="[A-Z]{2}[0-9]{4}"
       title="Use two uppercase letters followed by four digits"
       required>
```

Question 7: Create a file upload form

Solution

```
<form action="/upload"
      method="post"
      enctype="multipart/form-data">

    <label for="document">Upload PDF:</label>

    <input type="file"
           id="document"
           name="document"
           accept=".pdf,application/pdf"
           required>

    <button type="submit">Upload</button>
</form>
```

The server must support multipart requests and validate the uploaded file.

---

50. Important HTML Forms Viva Questions

1. What is an HTML form?

An HTML form collects information from users through controls such as input fields, checkboxes, radio buttons, and dropdowns.

2. What is the purpose of the `<form>` tag?

It groups form controls and defines how their data is submitted.

3. What is the difference between GET and POST?

GET normally places submitted form data in the URL query string, while POST generally sends it in the request body.

4. What is the purpose of the `action` attribute?

It specifies the URL that receives the submitted form data.

5. What is the purpose of the `method` attribute?

It specifies the HTTP method used for form submission.

6. What is the difference between `id` and `name`?

`id` identifies an element in the document, while `name` identifies the field in submitted form data.

7. What is the purpose of `<label>`?

It provides an accessible description for a form control.

8. What is the difference between radio buttons and checkboxes?

Radio buttons normally allow one selection per group. Checkboxes allow independent selections.

9. What is the purpose of the `required` attribute?

It makes a control required for normal browser constraint validation.

10. What is the difference between `readonly` and `disabled`?

Readonly controls cannot normally be edited but can generally be submitted. Disabled controls cannot normally be interacted with and are not submitted.

11. What is the purpose of the `placeholder` attribute?

It displays a short hint inside an empty control.

12. What is the purpose of the `maxlength` attribute?

It limits the maximum number of characters in supported text inputs.

13. What is the purpose of the `pattern` attribute?

It defines a regular expression used for client-side validation of supported text-like inputs.

14. What is the purpose of `<textarea>`?

It collects multiline text.

15. What is the purpose of `<select>`?

It creates a dropdown or selection list.

16. What is the purpose of `<fieldset>`?

It groups related form controls.

17. What is the purpose of `<legend>`?

It provides a caption for a fieldset.

18. What is the purpose of the `enctype` attribute?

It specifies how submitted form data is encoded, particularly for POST requests.

19. Why use `multipart/form-data`?

It is commonly used when submitting files through an HTML form.

20. What is the purpose of `autocomplete`?

It allows the browser to offer or manage suggested values for form fields.

21. What is the purpose of the `form` attribute on an input?

It associates a control with a form by the form's ID, even if the control is outside the form element.

22. What is the purpose of `formaction`?

It overrides the form's submission URL for a particular submit control.

23. What is the purpose of `formmethod`?

It overrides the form's submission method for a particular submit control.

24. What is the purpose of `formnovalidate`?

It allows a particular submit control to bypass the browser's built-in constraint validation.

25. Does HTML validation replace backend validation?

No. Client-side validation improves usability, but the server must validate incoming data because client-side checks can be bypassed.

Final Revision Checklist

- [ ] Understand the purpose of HTML forms.
- [ ] Use `action` and `method` correctly.
- [ ] Explain GET vs. POST.
- [ ] Understand form encoding and file uploads.
- [ ] Connect labels to controls.
- [ ] Use text, email, password, number, date, and file inputs.
- [ ] Understand radio buttons and checkboxes.
- [ ] Create dropdowns and textareas.
- [ ] Use fieldsets and legends.
- [ ] Apply required, minlength, maxlength, min, max, step, and pattern.
- [ ] Understand readonly vs. disabled.
- [ ] Understand form, formaction, formenctype, formmethod, formnovalidate, and formtarget.
- [ ] Build a complete student registration form.

- [ ] Understand that real submission requires a suitable backend.
- [ ] Validate submitted data on the server.

Recommended learning order

1. Start with simple forms and labels.
2. Practice input types.
3. Learn validation attributes.
4. Practice GET and POST.
5. Learn file uploads and form-specific submit attributes.
6. Build the student registration project.
7. Later, connect the form to a backend using a technology such as Java, Python Flask, PHP, or Node.js.